МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
УКРАЇНСЬКИЙ ДЕРЖАВНИЙ УНІВЕРСИТЕТ НАУКИ І ТЕХНОЛОГІЙ
Український державний хіміко-технологічний університет

Лабораторна робота №4
з дисципліни «Інформаційні технології в наукових дослідженнях»
Тема: «ВИЯВЛЕННЯ ОБ'ЄКТІВ ЗАДАНИХ КЛАСІВ НА ЗОБРАЖЕННЯХ З ВИКОРИСТАННЯМ ЗГОРТКОВИХ НЕЙРОННИХ МЕРЕЖ»
(Варіант 21: пульт і мобільний телефон, модель YOLO26)

Перевірила:
д.т.н., проф., Гнатушенко В.В.
«___» _____ 2026

Виконав:
аспірант групи G3
Ровенський Олег

Дніпро
2026

Зміст

1. Мета роботи 3
2. Завдання 3
3. Завдання 1 3
4. Завдання 2 5
5. Завдання 3 8
6. Лістинг коду 10
7. Висновки 10
8. Відповіді на контрольні питання 11
Література 12
Додаток А. Лістинг коду 14

1. Мета роботи

Вивчити принципи та інструменти програмної реалізації виявлення об'єктів заданих класів на зображеннях з використанням існуючих фреймворків, що реалізують згорткові нейронні мережі.

2. Завдання

1. Реалізувати виявлення об'єктів на зображеннях за допомогою згорткової нейронної мережі YOLO.
2. Порахувати точність і повноту детектора на 10 зображеннях з MS COCO для різних порогів впевненості. Кожне зображення має містити щонайменше 3 об'єкти різних класів.
3. Змінити код так, щоб виявлялися тільки класи за варіантом. Показати приклади на зображеннях з 5 і більше класами.

Вибір варіанта. Згідно з алфавітним порядком прізвища «Ровенський» (літера «Р» - 21-ша в українській абетці) обрано **варіант 21**: пульт (remote) і мобільний телефон (cell phone).

У методичці запропоновано YOLOv4. За погодженням з викладачем я використав новішу модель YOLO26 від Ultralytics (yolo26m.pt, навчена на MS COCO, 80 класів). На відміну від YOLOv4, YOLO26 не використовує anchor-рамки і видає одну рамку на об'єкт, тому окремий NMS не потрібен.

Використано Python 3.14, Ultralytics 8.4.208, PyTorch 2.15.0 і OpenCV. Зображення і розмітку взяв з набору MS COCO val2017.

Код: https://github.com/OlehRovenskyi/yolo26-coco-object-detection

3. Завдання 1

3.1. Модель

Модель завантажується одним рядком, ваги yolo26m.pt бібліотека завантажує сама. Розмір входу 640×640 (кратний 32). Для кожного зображення модель повертає рамки з класом і впевненістю:

```
model = YOLO("yolo26m.pt")
result = model.predict(path, conf=0.5, imgsz=640)[0]
for box, cls, score in zip(result.boxes.xyxy, result.boxes.cls, result.boxes.conf):
    print(model.names[int(cls)], float(score), box.tolist())
cv2.imwrite(out_path, result.plot())  # зображення з рамками
```

3.2. Результати

Запустив модель на 10 зображеннях з порогом 0,5. Вона знайшла 38 об'єктів. Два приклади на рисунках 1 і 2, решта в папці results/task1.



Рисунок 1 - Вантажівка, мотоцикли і людина (зображення 2)

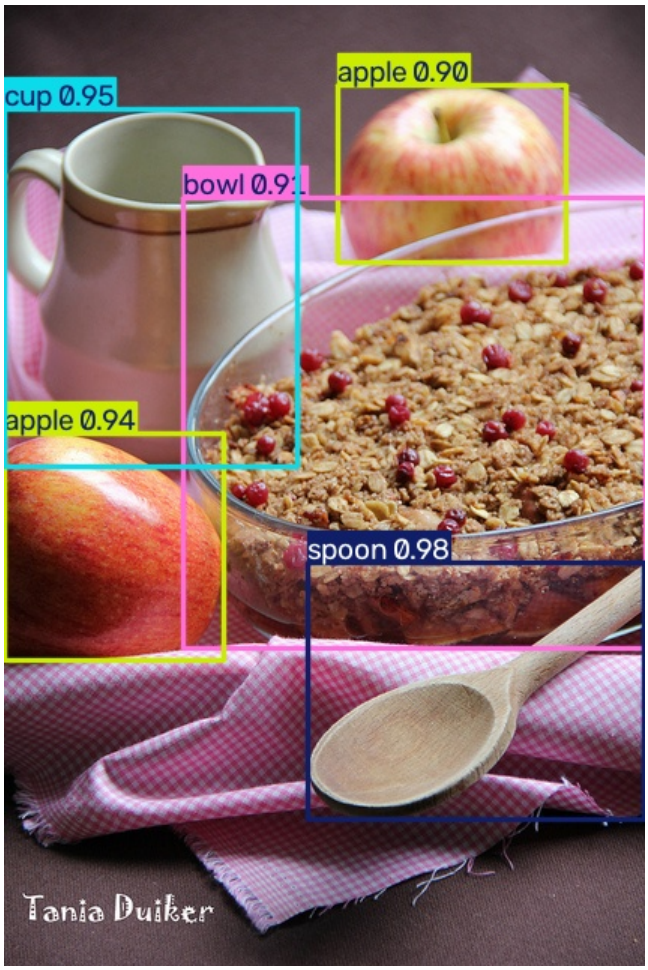


Рисунок 2 - Ложка, чашка, яблука і миска (зображення 4)

4. Завдання 2

4.1. Вибір зображень

Вибрав випадково 10 зображень з COCO val2017, на яких є щонайменше 3 класи і не більше 12 об'єктів. Зерно генератора фіксоване (seed = 42), тому при кожному запуску вибираються ті самі зображення (табл. 1).

Таблиця 1 - Зображення для оцінки

№	Файл COCO	Об'єктів	Класи
1	000000476810.jpg	9	bed, book, cat, remote, tv
2	000000081394.jpg	6	car, motorcycle, person, truck
3	000000016598.jpg	3	cell phone, person, tie
4	000000562059.jpg	6	apple, bowl, cup, dining table, spoon
5	000000211120.jpg	6	bowl, teddy bear, truck
6	000000182417.jpg	8	bowl, cake, cup, dining table, spoon
7	000000170099.jpg	3	couch, person, tie
8	000000099054.jpg	6	airplane, person, truck
9	000000559513.jpg	5	bottle, dining table, fork, knife
10	000000072852.jpg	4	bicycle, handbag, person

4.2. Як рахувалися p і r

Кожну рамку моделі порівнював з розміткою COCO. Рамка правильна (TP), якщо клас збігається і IoU ≥ 0,5. Рамки без пари - хибні виявлення (FP), об'єкти розмітки без пари - пропуски (FN).

$$p = TP / (TP + FP), \quad r = TP / (TP + FN), \quad F1 = 2 \cdot p \cdot r / (p + r)$$

4.3. Результати

Пороги від 0,3 до 0,9 з кроком 0,2. Прочерк у таблиці означає, що модель нічого не знайшла і точність порахувати не можна.

Таблиця 2 - Точність (p) і повнота (r), зображення 1-5

Поріг	1		2		3		4		5	
	p	r	p	r	p	r	p	r	p	r
0,3	0,75	0,33	0,80	0,67	1,00	1,00	1,00	1,00	0,67	1,00
0,5	1,00	0,22	0,75	0,50	1,00	1,00	1,00	0,83	0,83	0,83
0,7	1,00	0,22	1,00	0,17	1,00	0,67	1,00	0,83	1,00	0,83
0,9	1,00	0,22	1,00	0,17	1,00	0,33	1,00	0,67	1,00	0,50

Таблиця 3 - Точність (p) і повнота (r), зображення 6-10

Поріг	6		7		8		9		10	
	p	r	p	r	p	r	p	r	p	r
0,3	0,71	0,62	1,00	1,00	1,00	0,83	0,33	0,40	0,75	0,75
0,5	0,80	0,50	1,00	1,00	1,00	0,67	0,67	0,40	1,00	0,75
0,7	1,00	0,38	1,00	1,00	1,00	0,50	1,00	0,40	1,00	0,75
0,9	1,00	0,12	1,00	1,00	-	0,00	-	0,00	1,00	0,25

Таблиця 4 - Разом по 10 зображеннях

Поріг	TP	FP	FN	p	r	F1
0,3	40	12	16	0,77	0,71	0,74
0,5	34	4	22	0,89	0,61	0,72
0,7	29	0	27	1,00	0,52	0,68
0,9	16	0	40	1,00	0,29	0,44

Чим вищий поріг, тим вища точність і нижча повнота. При порозі 0,9 модель не помиляється (p = 1,00), але знаходить менше третини об'єктів (r = 0,29). Найкращий F1 = 0,74 при порозі 0,3. Його використав у завданні 3.

4.4. Найгірші результати

Три зображення з найменшим F1 при порозі 0,3. Синя рамка - правильне виявлення (TP), червона - хибне (FP), жовта - пропущений об'єкт (FN).

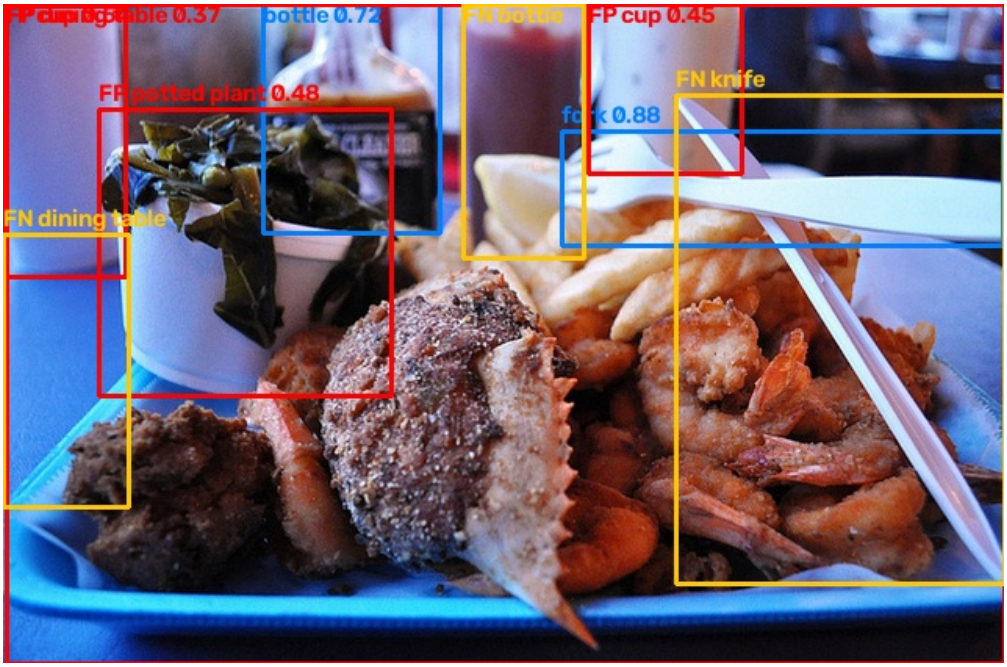


Рисунок 3 - Зображення 9: TP = 2, FP = 4, FN = 3

Миску із зеленню модель прийняла за рослину в горщику, а синій піднос за обідній стіл. Пластиковий ніж закритий виделкою та їжею, його пропущено. Одна зі «хибних» чашок, схоже, справжня, просто її немає в розмітці COCO.

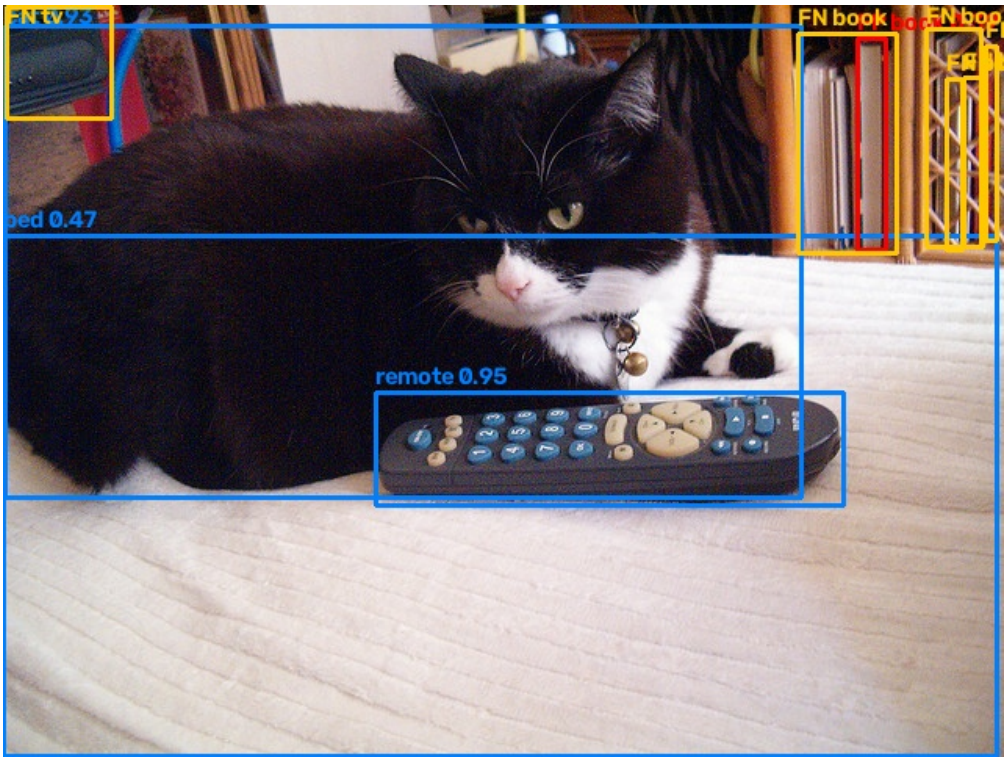


Рисунок 4 - Зображення 1: TP = 3, FP = 1, FN = 6

Кота, пульт і ліжко знайдено правильно. Пропущено книжки на полиці: вони вузькі (7-34 пікселі) і стоять щільно. Телевізор у кутку теж малий і обрізаний краєм кадру.

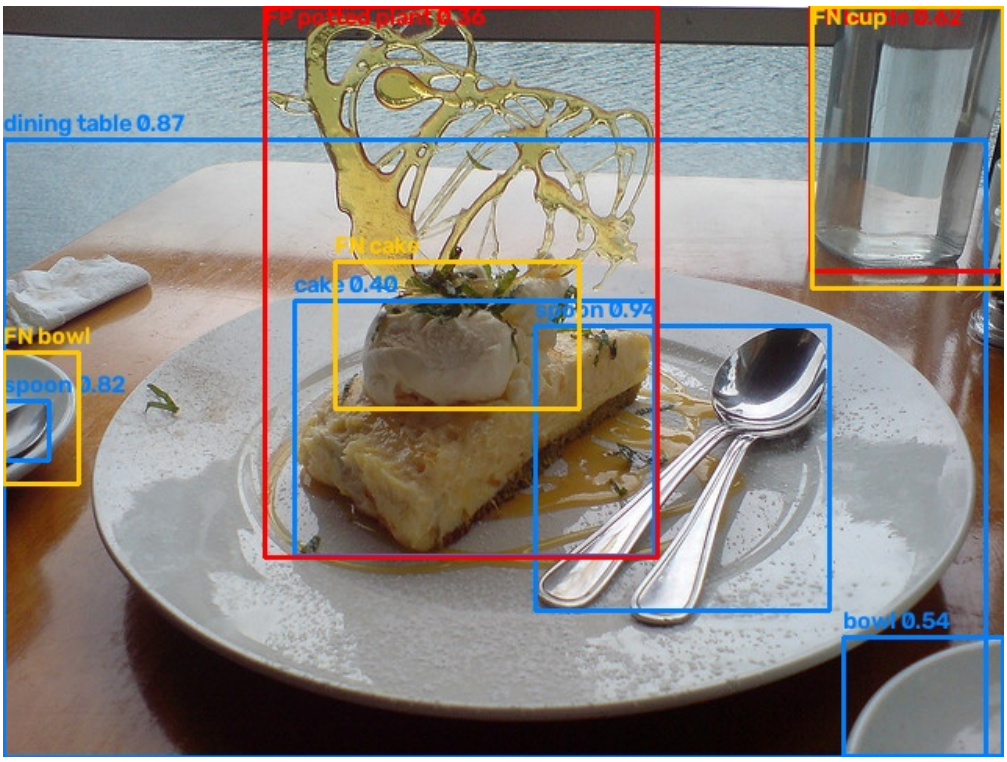


Рисунок 5 - Зображення 6: TP = 5, FP = 2, FN = 3

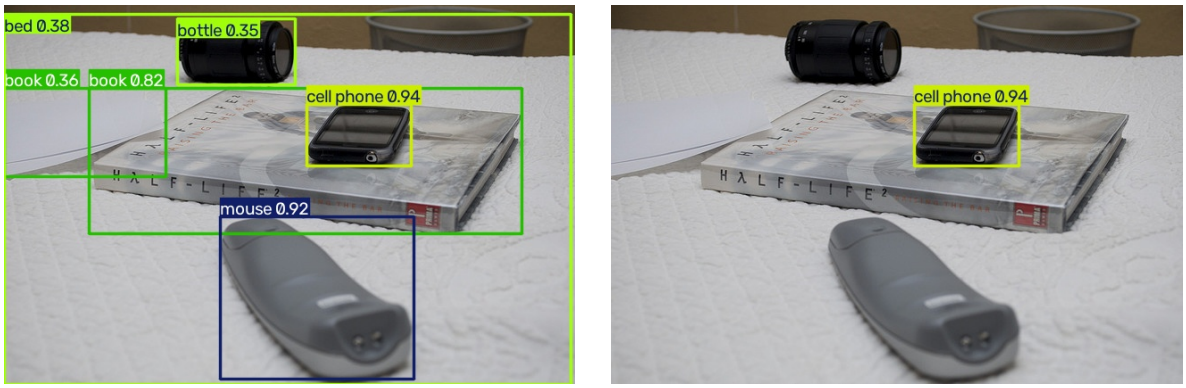
Склянку з водою модель прийняла за пляшку, тому одна помилка дала і FP, і FN. Карамельну прикрасу прийнято за рослину. Крем на торті в COCO розмічено як окремий торт, а модель знайшла торт однією рамкою.

5. Завдання 3

Варіант 21: пульт (remote) і мобільний телефон (cell phone). У методичці це класи 66 і 68, а в моделі нумерація з 0, тому індекси 65 і 67. Замість умови if class_id == … використав вбудований фільтр Ultralytics - параметр classes. Поріг 0,3, найкращий із завдання 2:

```
ids = [k for k, v in model.names.items() if v in ["remote", "cell phone"]]  # [65, 67]
result = model.predict(path, conf=0.3, imgsz=640, classes=ids)[0]
```

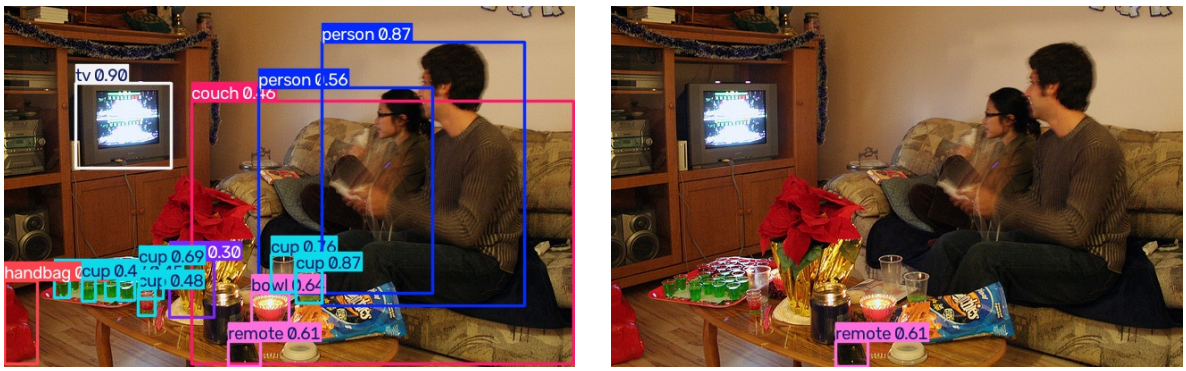
Вибрав 4 зображення з COCO val2017, на яких є і пульт, і телефон, а всього від 5 до 9 класів (рисунки 6-9).



а) усі класи *б) лише remote і cell phone*

Рисунок 6 - Варіант 21, зображення 1 (000000396729.jpg)

Телефон знайдено (0,94). Пульт незвичної обтічної форми модель прийняла за мишу (mouse 0,92), тому з фільтром він зник.



а) усі класи *б) лише remote і cell phone*

Рисунок 7 - Варіант 21, зображення 2 (000000036936.jpg)

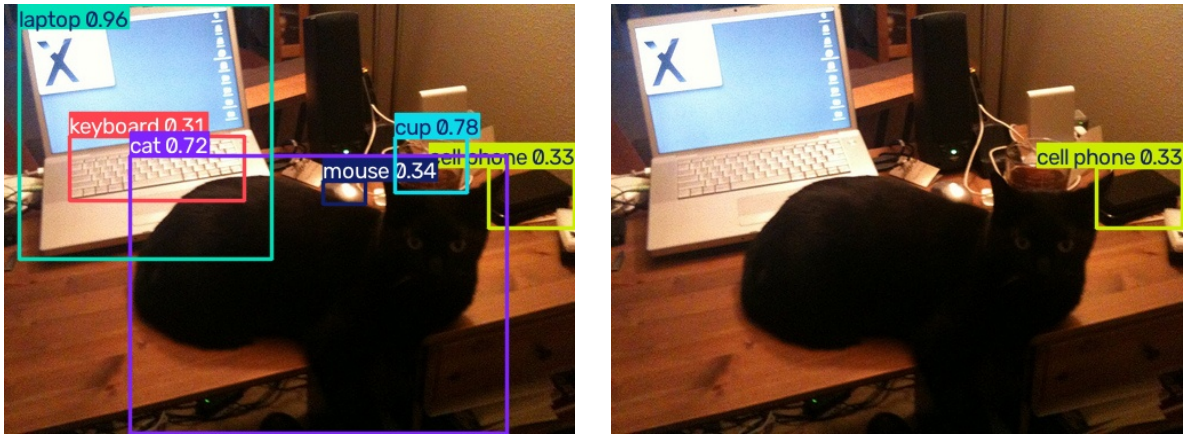
Знайдено один пульт (0,61). Другий пульт (19×17 пікселів) і телефон (35×28 пікселів) надто малі, їх пропущено.



а) усі класи *б) лише remote і cell phone*

Рисунок 8 - Варіант 21, зображення 3 (000000026465.jpg)

Телефон знайдено (0,93). Пульт модель прийняла за телефон (cell phone 0,82).



а) усі класи *б) лише remote і cell phone*

Рисунок 9 - Варіант 21, зображення 4 (000000119828.jpg)

Пульт (16×39) і телефон (14×44) дуже малі, їх пропущено. Рамка cell phone 0,33 праворуч хибна.

Фільтр працює: праворуч залишаються тільки пульти і телефони. Але фільтр не виправляє помилки моделі. Якщо пульт розпізнано як мишу, з фільтром його не буде. Телефони модель знаходить добре, пульти гірше, а об'єкти до 40 пікселів здебільшого пропускає.

6. Лістинг коду

Код: https://github.com/OlehRovenskyi/yolo26-coco-object-detection

Структура проєкту:

```
yolo26-coco-object-detection/
├── data/
│   ├── annotations/           розмітка COCO val2017
│   └── val2017/               вибрані зображення COCO
├── results/
│   ├── task1/                 результати завдання 1
│   ├── task2/conf_0.3 … 0.9/  результати завдання 2 для кожного порогу
│   ├── task3/                 результати завдання 3
│   ├── task1..3_report.md     звіти з таблицями
│   └── IT_НД_лаб_4_….docx       звіт до лабораторної роботи
├── lab4.py                    код усіх трьох завдань
├── README.md                  як встановити і запустити
├── requirements.txt           залежності Python
└── yolo26m.pt                 ваги моделі (завантажуються автоматично)
```

Код наведено в додатку А (lab4.py). Як встановити і запустити - у README.md.

7. Висновки

YOLO26 запускається кількома рядками коду і знаходить об'єкти 80 класів COCO.

Поріг впевненості - це вибір між точністю і повнотою. При 0,3 маємо p = 0,77 і r = 0,71, при 0,9 p = 1,00, але r = 0,29. Найкращий F1 = 0,74 при порозі 0,3.

Модель пропускає малі, щільно розташовані, закриті іншими предметами або обрізані краєм кадру об'єкти. Помиляється на схожих класах (склянка і пляшка, пульт і миша чи телефон). Частина «помилок» насправді через неповну розмітку COCO.

Для варіанту 21 фільтр classes залишає тільки пульти і телефони. Щоб краще знаходити малі об'єкти, можна збільшити розмір входу (наприклад, 1280) або взяти більшу модель (yolo26l, yolo26x).

8. Відповіді на контрольні питання

1. Склад моделі штучного нейрона?

Входи x1…xn, ваги w (синапси), зміщення b, суматор і функція активації φ. Нейрон рахує зважену суму входів, додає зміщення і пропускає результат через функцію активації: $y = \varphi(b + \Sigma\, wi \cdot xi)$.

2. Призначення та приклади функцій активації?

Функція активації обмежує вихід нейрона і робить перетворення нелінійним. Без неї мережа з будь-якою кількістю шарів була б лінійною. Приклади: сигмоїда, tanh, ReLU, Leaky ReLU, PReLU, Mish (у YOLOv4), SiLU, softmax (на виході класифікатора).

3. Топологія СНР для багатокласової класифікації? Сформулюйте призначення кожного типу шару СНР.

Вхід -> згорткові шари -> активація ReLU -> шари підвибірки (pooling) -> flatten -> повнозв'язні шари -> softmax. Згортковий шар виділяє ознаки (межі, текстури, частини об'єктів). Шар підвибірки зменшує розмір карт ознак і робить мережу стійкішою до малих зсувів. Flatten перетворює карти ознак у вектор. Повнозв'язні шари поєднують ознаки, а softmax дає ймовірність кожного класу.

4. Наведіть приклад обчислення згортки для фрагмента зображення та довільного ядра шару.

Фрагмент 3×3 і ядро для пошуку вертикальних меж:

```
фрагмент          ядро
10  20  30       -1  0  1
40  50  60       -1  0  1
70  80  90       -1  0  1
```

Множимо поелементно і додаємо: (−10 + 30) + (−40 + 60) + (−70 + 90) = 20 + 20 + 20 = 60. Зі зміщенням b = 0 вихід 60. Якщо зображення 4×4, ядро 3×3 проходить його з кроком 1 і дає карту ознак 2×2 (4 − 3 + 1 = 2).

5. Що розуміється під завданням виявлення об'єктів на зображенні?

Знайти на зображенні всі об'єкти заданих класів: для кожного визначити клас і рамку (положення і розмір). Скільки об'єктів на зображенні, наперед невідомо.

6. Які критерії можуть бути використані для оцінки ефективності виявлення об'єктів на зображеннях з використанням СНР?

IoU (перекриття рамки з еталоном), точність p, повнота r, F1, середня точність AP по класу і mAP по всіх класах (наприклад, mAP@0.5 або mAP@0.5:0.95). Для роботи в реальному часі важлива ще швидкість (кадрів за секунду).

7. Які сучасні моделі СНР застосовуються для виявлення об'єктів на зображенні? Дайте визначення однопрохідним та двопрохідним моделям СНР.

Двопрохідні: R-CNN, Fast R-CNN, Faster R-CNN, Mask R-CNN. Спочатку знаходять регіони, де можуть бути об'єкти, потім класифікують кожен регіон. Точні, але повільніші. Однопрохідні: YOLO (v3, v4, v5, v8, YOLO11, YOLO26), SSD, RetinaNet. Знаходять і класифікують об'єкти за один прохід мережі, тому швидші. Є також моделі на трансформерах: DETR, RT-DETR.

8. У чому переваги моделей сімейства Yolo?

Висока швидкість, працюють у реальному часі. Одна мережа робить і локалізацію, і класифікацію. Мережа бачить усе зображення, тому рідше плутає фон з об'єктом. Є моделі різного розміру, від маленьких для телефонів до великих і точних. Їх легко запустити і донавчити.

9. Який фреймворк підтримує використання Yolo для детектування об'єктів і які особливості його застосування?

Darknet - оригінальний фреймворк YOLO (C і CUDA), у ньому навчають YOLOv4. OpenCV (модуль dnn) читає cfg і weights Darknet через readNetFromDarknet. Він тільки запускає готову модель, NMS і розбір виходів треба писати самому, розмір входу кратний 32. Ultralytics (PyTorch) підтримує YOLOv5, YOLOv8, YOLO11 і YOLO26. Ставиться через pip, модель запускається одним викликом predict, є навчання, фільтр класів і експорт в інші формати. У цій роботі використано Ultralytics.

10. З яких основних блоків складається СНР YOLOv4?

Блок вилучення ознак (backbone) - CSPDarknet-53 з активацією Mish. Блок збору карт ознак з різних шарів (neck) - SPP і PAN. Блок виявлення і класифікації (head) - формує карти ознак на трьох масштабах (кроки 8, 16, 32) і для кожної комірки передбачає рамки, впевненість і клас.

Література

1. Ultralytics YOLO26 [Electronic resource]. - Mode of access: https://docs.ultralytics.com/models/yolo26/
2. Ultralytics [Electronic resource]. - Mode of access: https://github.com/ultralytics/ultralytics
3. COCO - Common Objects in Context [Electronic resource]. - Mode of access: https://cocodataset.org/
4. Lin T.-Y. et al. Microsoft COCO: Common Objects in Context // ECCV. - 2014.
5. Bochkovskiy A., Wang C.-Y., Liao H.-Y. M. YOLOv4: Optimal Speed and Accuracy of Object Detection [Electronic resource]. - Mode of access: https://arxiv.org/abs/2004.10934

6. Darknet [Electronic resource]. - Mode of access: https://github.com/AlexeyAB/darknet

Додаток А. Лістинг коду

lab4.py

```
"""Lab 4: object detection on MS COCO images with YOLO26.

Task 1: detect objects of all 80 COCO classes with the YOLO26 model (Ultralytics).
Task 2: precision (p) and recall (r) on 10 COCO val2017 images for confidence
thresholds 0.3..0.9.
Task 3: detect only the classes of variant 21 (remote and cell phone).
"""

import json
import random
import urllib.request
from collections import defaultdict
from pathlib import Path

import cv2
from ultralytics import YOLO

ROOT = Path(__file__).resolve().parent
IMAGES = ROOT / "data" / "val2017"
ANNOTATIONS = ROOT / "data" / "annotations" / "instances_val2017.json"
RESULTS = ROOT / "results"

MODEL = "yolo26m.pt"
IMAGE_SIZE = 640
TASK1_CONF = 0.5
THRESHOLDS = [0.3, 0.5, 0.7, 0.9]
IOU_MATCH = 0.5
NUM_IMAGES = 10
SEED = 42

VARIANT = 21
VARIANT_CLASSES = ["remote", "cell phone"]
VARIANT_IMAGES = 4

TP_COLOR, FP_COLOR, FN_COLOR = (255, 128, 0), (0, 0, 255), (0, 200, 255)


def load_coco():
    """Returns image info by id and, for each image, its objects as {name, box} plus a
crowd flag."""
    with open(ANNOTATIONS) as f:
        coco = json.load(f)
    names = {c["id"]: c["name"] for c in coco["categories"]}
    images = {img["id"]: img for img in coco["images"]}
    objects, crowd = defaultdict(list), set()
    for a in coco["annotations"]:
        x, y, w, h = a["bbox"]
        objects[a["image_id"]].append({"name": names[a["category_id"]], "box": [x, y,
x + w, y + h]})
        if a["iscrowd"]:
            crowd.add(a["image_id"])
    return images, objects, crowd


def select(objects, crowd, condition, count):
    # "crowd" annotations cover a group of objects with one box, so they can't be
matched fairly
    candidates = sorted(i for i, objs in objects.items() if i not in crowd and
condition(objs))
    return random.Random(SEED).sample(candidates, count)


def classes_of(objs):
    return {o["name"] for o in objs}


def download(images, image_ids):
```

```
    IMAGES.mkdir(parents=True, exist_ok=True)
    for image_id in image_ids:
        path = IMAGES / images[image_id]["file_name"]
        if not path.exists():
            urllib.request.urlretrieve(images[image_id]["coco_url"], path)
    return {i: IMAGES / images[i]["file_name"] for i in image_ids}


def predict(model, path, conf, classes=None):
    result = model.predict(str(path), conf=conf, classes=classes, imgsz=IMAGE_SIZE,
verbose=False)[0]
    boxes = [
        {"name": model.names[int(c)], "conf": float(s), "box": b.tolist()}
        for b, c, s in zip(result.boxes.xyxy, result.boxes.cls, result.boxes.conf)
    ]
    return result, boxes


def iou(a, b):
    ix = max(0.0, min(a[2], b[2]) - max(a[0], b[0]))
    iy = max(0.0, min(a[3], b[3]) - max(a[1], b[1]))
    inter = ix * iy
    union = (a[2] - a[0]) * (a[3] - a[1]) + (b[2] - b[0]) * (b[3] - b[1]) - inter
    return inter / union if union > 0 else 0.0


def match(predictions, truth):
    """Greedy matching: a box is TP if an unmatched ground-truth object of the same
class has IoU >= 0.5."""
    used, tp, fp = set(), [], []
    for p in sorted(predictions, key=lambda p: -p["conf"]):
        candidates = [(iou(p["box"], t["box"]), i) for i, t in enumerate(truth)
                      if i not in used and t["name"] == p["name"]]
        best_iou, best = max(candidates, default=(0.0, None))
        if best_iou >= IOU_MATCH:
            used.add(best)
            tp.append(p)
        else:
            fp.append(p)
    return tp, fp, [t for i, t in enumerate(truth) if i not in used]


def metrics(tp, fp, fn):
    """p, r and F1; None where the denominator is zero."""
    def div(a, b):
        return a / b if b else None
    return div(tp, tp + fp), div(tp, tp + fn), div(2 * tp, 2 * tp + fp + fn)


def fmt(value):
    return "-" if value is None else f"{value:.2f}"


def table(header, rows):
    return ["| " + " | ".join(header) + " |", "|" + "---|" * len(header)] + \
           ["| " + " | ".join(str(c) for c in row) + " |" for row in rows]


def describe(items):
    return "; ".join(
        i["name"] + (f" ({i['conf']:.2f})" if "conf" in i else "")
        + f", {int(i['box'][2] - i['box'][0])}×{int(i['box'][3] - i['box'][1])} px"
        for i in items
    ) or "none"


def draw_matches(path, out, tp, fp, fn):
    image = cv2.imread(str(path))
    for items, color, prefix in [(tp, TP_COLOR, ""), (fp, FP_COLOR, "FP "), (fn,
FN_COLOR, "FN ")]:
        for item in items:
            x1, y1, x2, y2 = (int(v) for v in item["box"])
```

```
            label = prefix + item["name"] + (f" {item['conf']:.2f}" if "conf" in item
else "")
            cv2.rectangle(image, (x1, y1), (x2, y2), color, 2)
            cv2.putText(image, label, (x1, max(y1 - 5, 12)), cv2.FONT_HERSHEY_SIMPLEX,
0.5, color, 2)
    cv2.imwrite(str(out), image)


def write(name, lines):
    (RESULTS / name).write_text("\n".join(lines) + "\n")


def task1(model, images, paths, image_ids):
    out_dir = RESULTS / "task1"
    out_dir.mkdir(parents=True, exist_ok=True)
    lines = ["# Task 1. Object detection with YOLO26", "",
             f"Model `{MODEL}` (Ultralytics, trained on MS COCO, {len(model.names)}
classes), "
             f"input size {IMAGE_SIZE}×{IMAGE_SIZE}, confidence threshold
{TASK1_CONF}.", "",
             "## Detected objects (class, confidence, box size)", ""]
    for n, image_id in enumerate(image_ids, start=1):
        result, found = predict(model, paths[image_id], TASK1_CONF)
        cv2.imwrite(str(out_dir / f"{n:02d}_{image_id}.jpg"), result.plot())
        lines.append(f"- Image {n} (`{images[image_id]['file_name']}`, {len(found)}
objects): {describe(found)}")
    write("task1_report.md", lines)


def task2(model, images, objects, paths, image_ids):
    """Returns the best threshold by F1 over all images."""
    stats = {}  # (image number, threshold) -> (tp, fp, fn) lists
    for n, image_id in enumerate(image_ids, start=1):
        _, predictions = predict(model, paths[image_id], min(THRESHOLDS))
        for th in THRESHOLDS:
            out_dir = RESULTS / "task2" / f"conf_{th}"
            out_dir.mkdir(parents=True, exist_ok=True)
            stats[n, th] = match([p for p in predictions if p["conf"] >= th],
objects[image_id])
            draw_matches(paths[image_id], out_dir / f"{n:02d}_{image_id}.jpg",
*stats[n, th])

    def counts(n, th):
        return [len(x) for x in stats[n, th]]

    numbers = range(1, len(image_ids) + 1)
    totals = {th: [sum(c) for c in zip(*(counts(n, th) for n in numbers))] for th in
THRESHOLDS}
    best = max(THRESHOLDS, key=lambda th: metrics(*totals[th])[2] or 0)
    worst = sorted(numbers, key=lambda n: metrics(*counts(n, best))[2] or 0)[:3]

    lines = ["# Task 2. Precision and recall of the YOLO26 detector", "", "## Images",
"",
             *table(["#", "COCO file", "Objects", "Classes"],
                    [[n, images[i]["file_name"], len(objects[i]), ",
".join(sorted(classes_of(objects[i])))]
                     for n, i in zip(numbers, image_ids)]),
             "", "## p / r table", "",
             *table(["Confidence"] + [f"{n} {m}" for n in numbers for m in ("p",
"r")],
                    [[th] + [fmt(v) for n in numbers for v in metrics(*counts(n, th))
[:2]] for th in THRESHOLDS]),
             "", f"## Totals over all {len(image_ids)} images", "",
             *table(["Confidence", "TP", "FP", "FN", "p", "r", "F1"],
                    [[th, *totals[th], *map(fmt, metrics(*totals[th]))] for th in
THRESHOLDS]),
             "", f"Best threshold by F1: **{best}**", "", f"## Three worst results
(threshold {best})", ""]
    for n in worst:
        image_id = image_ids[n - 1]
        tp, fp, fn = stats[n, best]
        lines += [f"### Image {n} ({images[image_id]['file_name']})", "",
```

```
                  f"File: `results/task2/conf_{best}/{n:02d}_{image_id}.jpg`", "",
                  f"- TP: {len(tp)}, FP: {len(fp)}, FN: {len(fn)}",
                  f"- False detections (FP): {describe(fp)}",
                  f"- Missed objects (FN): {describe(fn)}", ""]
    lines.append("Colors on the images: blue is a correct detection (TP), red is a
false detection (FP), "
                 "yellow is a missed object from the COCO annotation (FN).")
    write("task2_report.md", lines)

    for th in THRESHOLDS:
        p, r, f1 = metrics(*totals[th])
        print(f"conf={th}: p={fmt(p)} r={fmt(r)} F1={fmt(f1)}")
    return best


def task3(model, images, objects, paths, image_ids, conf):
    out_dir = RESULTS / "task3"
    out_dir.mkdir(parents=True, exist_ok=True)
    ids = [k for k, v in model.names.items() if v in VARIANT_CLASSES]
    lines = [f"# Task 3. Variant {VARIANT}: {' and '.join(VARIANT_CLASSES)}", "",
             f"Model class ids (numbered from 0): {ids}. Confidence threshold:
{conf}.", ""]
    for n, image_id in enumerate(image_ids, start=1):
        result_all, found_all = predict(model, paths[image_id], conf)
        result_variant, found_variant = predict(model, paths[image_id], conf,
classes=ids)
        cv2.imwrite(str(out_dir / f"{n:02d}_all_classes.jpg"), result_all.plot())
        cv2.imwrite(str(out_dir / f"{n:02d}_variant_only.jpg"), result_variant.plot())
        truth = objects[image_id]
        lines += [f"## Image {n} ({images[image_id]['file_name']})", "",
                  f"- Classes in the COCO annotation ({len(classes_of(truth))}): {',
'.join(sorted(classes_of(truth)))}",
                  f"- Annotated objects of the variant classes: {describe(o for o in
truth if o['name'] in VARIANT_CLASSES)}",
                  f"- All detections without the filter ({len(found_all)}):
{describe(found_all)}",
                  f"- Detections with the variant filter ({len(found_variant)}):
{describe(found_variant)}",
                  f"- Files: `results/task3/{n:02d}_all_classes.jpg`, `results/task3/
{n:02d}_variant_only.jpg`", ""]
    write("task3_report.md", lines)


def main():
    RESULTS.mkdir(exist_ok=True)
    images, objects, crowd = load_coco()
    task2_ids = select(objects, crowd, lambda objs: len(classes_of(objs)) >= 3 and
len(objs) <= 12, NUM_IMAGES)
    task3_ids = select(objects, crowd, lambda objs: set(VARIANT_CLASSES) <=
classes_of(objs)
                       and len(classes_of(objs)) >= 5, VARIANT_IMAGES)
    paths = download(images, task2_ids + task3_ids)

    model = YOLO(MODEL)
    task1(model, images, paths, task2_ids)
    best = task2(model, images, objects, paths, task2_ids)
    task3(model, images, objects, paths, task3_ids, best)
    print(f"Best threshold: {best}. Reports: {RESULTS}")


if __name__ == "__main__":
    main()
```